

CS201 Portal 前后端架构小白说明

从浏览器页面、React 组件、Next.js API、老师站同步、本地文件到部署测试的完整讲解

生成日期：2026-05-30。事实来源：当前 cs201-portal 工作区、项目代码、.agentdocs 架构文档和部署文档。

说明：阅读承诺：这份文档不写 .env.local 的真实密码、token、账号密码明文，也不复制私有课程文件内容。文档只解释结构、原理、数据流和维护入口。

0. 先把整件事讲成人话

如果只用一句话解释这个网站：CS201 Portal 是一个课程门户网站。学生在浏览器里看到课程周计划、作业、Quiz、Reflection、Projects、Resources 等页面；服务器负责登录、保护资料、读取本地课程数据、和老师站同步每个人自己的进度；管理员页面负责把本地展示用的日期、可见性和 Class Notes 文件整理好。

这不是一个只有 HTML 的静态页面，也不是一个前端仓库加一个独立 Express 或 Django 后端。它使用 Next.js，所以页面和 API 写在同一个项目里。你可以把它理解成一个放在同一栋楼里的前台和办公室：前台是用户能看到的页面，办公室是服务器上处理登录、数据、同步和文件的后端逻辑。

第一次读这份文档时，不要从代码细节开始。先建立三个问题：用户点了哪里，浏览器向哪里请求，服务器最后从哪里拿数据。只要能回答这三个问题，大多数前后端结构就能讲清楚。

0.1 最小心智模型

1. 浏览器负责展示和交互。它运行 React 组件，让按钮、列表、表格、搜索和主题切换变成真实界面。
2. Next.js 服务器负责可信逻辑。它检查 cookie、读取本地 JSON、访问老师站 API、处理上传文件，并返回安全处理后的数据。
3. 本地文件负责保存课程材料和管理员覆盖数据。例如 data/course-site/normalized.json、data/homework/manifest.json、data/admin-overrides/course-overrides.json。
4. 老师站是外部系统。Homework、Quiz、Reflection、BK Projects 这类个人状态不是凭空猜的，而是服务器端模块用当前用户身份去老师站读取或提交。

0.2 这份文档怎么读

1. 第 1 部分先讲概念，解决前端、后端、API、cookie、server-side、client-side 到底是什么意思。
2. 第 2 部分给项目地图，告诉你每个目录大概做什么。
3. 第 3 和第 4 部分分别讲前端和后端真实怎么工作。
4. 第 5 部分按业务流程走一遍，最适合用来给别人讲这个网站。
5. 第 6 和第 7 部分讲数据、安全、部署和验证，适合维护或排查问题时看。
6. 附录是词汇表和路径索引，用来快速查某个技术名词或文件路径。

1. 前端、后端和 API 到底是什么

1.1 前端不是美化页面那么简单

前端是用户在浏览器里直接看到并操作的那一层。对 CS201 Portal 来说，Navbar、Week Navigation、Home 的学习地图、Homework 的题目选择器、Resources 的资料列表、Admin 的管理表格，都属于前端。前端的任务不是单纯让页面好看，而是把服务器给的数据组织成用户能理解、能操作、不会迷路的界面。

前端通常由三类东西组成。HTML 决定页面上有什么元素，例如标题、链接、按钮、表格。CSS 决定这些元素长什么样，例如颜色、边框、间距、响应式布局。JavaScript 决定页面如何动起来，例如点 Week 3 后切换当前周，提交 Quiz 后刷新状态，搜索后隐藏不匹配的模块。

这个项目用 React 写前端。React 的思路是把页面拆成组件。组件可以理解成网页零件：Navbar 是导航零件，HomeworkPracticeBoard 是作业练习主面板，ResourcesBoard 是资料工作台，AdminBoard 是管理员工作台。页面不是一大坨代码，而是很多零件组合起来。

1.2 后端是浏览器看不到但必须可信的那层

后端是服务器上的逻辑。用户看不到后端代码，但每次登录、刷新作业状态、下载受保护文件、提交 Quiz、上传 Class Notes，背后都有后端在工作。后端的核心职责是保护安全边界：浏览器不能直接拿老师站 token，不能直接改服务器 JSON 文件，不能绕过权限下载受限课程材料。

在 CS201 Portal 里，后端入口主要是 `app/api/.../route.ts` 这些文件。比如 `app/api/homework/status/route.ts` 是 Homework 状态接口，`app/api/quiz/week/[week]/route.ts` 是 Quiz 周数据接口，`app/api/admin/overrides/route.ts` 是 Admin 保存覆盖数据的接口。

后端拿到请求后，通常不会把所有逻辑都写在 `route.ts` 里。`route.ts` 更像入口和门卫：先检查登录，再读参数，再调用 `lib/...` 里的业务模块。真正复杂的同步、解析、保存逻辑放在 `lib/auth`、`lib/homework`、`lib/quiz`、`lib/reflections`、`lib/admin`、`lib/class-notes` 等目录。

1.3 API 是前端和后端之间的对话格式

API 可以理解成浏览器和服务器约好的接口。前端不需要知道服务器内部怎么读 JSON、怎么请求老师站，只需要知道可以请求某个 URL，服务器会返回某种 JSON。比如前端请求 `/api/homework/status`，后端返回每道 JPA 题是 `correct`、`incorrect`、`ungraded` 还是 `not-started`。

场景	前端做什么	后端做什么
打开 Homework	渲染 HomeworkPracticeBoard，并请求当前用户状态	验证 cookie，必要时用老师站会话读取 JPA submissions
提交 Quiz	收集用户填写的答案和 submitContextId	用服务器端保存的 submit context 把答案发回老师站
上传 Class Notes	选择文件、显示队列和进度	检查文件类型，写入 <code>public/course-materials/class-notes/</code> 和 <code>data/admin-overrides/class-notes.json</code>
保存 Admin 日期	把表格里的标题、日期、时间、隐藏状态发给 API	检查管理员权限，清洗 payload，写入本地 override JSON

1.4 为什么 Next.js 让前端和后端看起来在一起

传统项目可能有两个仓库：一个 React 前端，一个 Express/Django/Spring 后端。这个项目使用 Next.js App Router，所以前端页面和后端 API 可以放在同一个 Next.js 项目里。`app/homework/page.tsx` 是页面，`app/api/homework/status/route.ts` 是 API。它们在一个仓库，但运行位置不同：页面组件最终服务给浏览器，API route 在服务器上执行。

说明：关键区别：同一个代码仓库不等于同一个安全边界。只要代码运行在浏览器里，就不能放秘密。只要代码运行在服务器 route 或 server-side lib 里，才可以读取 .env.local、teacher token、本地私有文件等敏感资源。

2. CS201 Portal 项目总览

2.1 技术栈一句话解释

当前项目核心技术栈是 Next.js 16 App Router、React 19、TypeScript、Tailwind CSS v4、Vitest 和 Playwright。Next.js 负责页面路由、API 路由和生产构建；React 负责组件化 UI；TypeScript 让数据结构更明确；Tailwind 和 globals.css 负责样式；Vitest 负责单元/组件测试；Playwright 负责真实浏览器端到端测试。

技术	在这个项目里做什么	小白理解
Next.js 16 App Router	app/ 目录下自动按文件夹生成页面和 API	网站骨架和服务器入口
React 19	components/ 里的组件拼出页面	网页零件系统
TypeScript	给函数、组件、payload 定义类型	提前发现数据形状不对
Tailwind CSS v4	配合 app/globals.css 和 token 统一样式	快速写一致的间距、颜色和布局
Vitest	跑函数和组件测试	检查小块逻辑是否正确
Playwright	打开真实浏览器验证页面	模拟用户真的点网页

2.2 目录地图

理解 Next.js 项目时，最重要的是先知道目录分工。app/ 是路由入口，components/ 是 UI 零件，providers/ 是跨页面共享状态，lib/ 是业务逻辑，data/ 是本地结构化数据，public/ 是浏览器可以访问的静态文件，scripts/ 是生成或导入数据的脚本，tests/ 是验证。

目录或文件	真实作用
app/layout.tsx	全站根布局。挂载 CourseUiProvider、Navbar 和 main 页面容器。
app/*/page.tsx	页面入口。例如 app/homework/page.tsx 对应 /homework。
app/api/*/route.ts	后端 API 入口。浏览器请求这些 URL 时，由服务器执行。
components/layout/Navbar.tsx	顶部导航、周选择、全局搜索、移动端抽屉和登出入口。
components/pages/*.tsx	各大页面的主体 UI，例如 Home、Homework、Resources、Projects、Admin。
providers/CourseUiProvider.tsx	全站共享 UI 状态，例如当前周、主题、搜索词、本地完成状态、收藏。
lib/*	业务模块。认证、课程数据、老师站同步、Admin 覆盖、Class Notes、Homework 状态都在这里。
data/*	本地 JSON 数据。包括课程站导入结果、作业 manifest、材料 manifest、管理员覆盖数据。
public/*	浏览器可访问的静态资源。课程材料、模板、Class Notes 文件最终通过这里服务给页面。

2.3 请求从浏览器到数据源的总路线

下面用纯编号写一次最通用的数据流，避免花哨图形影响阅读。大部分页面都能套进这个模型。

1. 用户在浏览器访问一个页面，例如 /homework。
2. Next.js 根据 app/homework/page.tsx 找到页面入口。

- 3. 页面入口渲染 `components/pages/HomeworkPracticeBoard.tsx` 这类 React 组件。
- 4. 组件需要动态数据时，会请求 `app/api/.../route.ts` 暴露的 API。
- 5. API route 先通过 `lib/auth/session.ts` 或 `proxy.ts` 验证当前用户是否登录。
- 6. API route 调用 `lib/...` 里的业务模块读取本地 JSON、读取 `public` 文件元数据，或通过服务器端会话访问老师站。
- 7. 服务器把清洗后的 JSON 返回给浏览器。浏览器只拿到展示需要的数据，不拿密码、token、老师站原始敏感响应。
- 8. React 组件根据 JSON 更新界面，例如显示 `correct/incorrect` 状态、资料列表或 Admin 保存结果。

2.4 这个项目最重要的边界

说明：最重要的边界是：用户浏览器永远不是可信环境。浏览器可以展示结果、发起请求、保存本地 UI 偏好，但不能保管老师站 token，不能直接写服务器文件，不能绕过 API 权限。所有真正敏感或可信的动作都必须在 Next.js API route 和 `server-side lib` 里完成。

3. 前端怎么工作

3.1 `app/layout.tsx` 是全站外壳

`app/layout.tsx` 是 Next.js App Router 的根布局。它定义 HTML、body、全局 metadata，导入 `app/globals.css`，并把所有页面 children 放进 `CourseUiProvider`、Navbar 和 `main.page-container` 里面。也就是说，不管你打开 Home、Homework、Resources 还是 Admin，都会先经过这个外壳。

这个文件的意义很大：它保证所有页面都有相同的顶部导航、相同的主题状态、相同的周选择状态、相同的页面容器。这样每个页面不需要重复写一套导航和全局状态。

3.2 `app/*/page.tsx` 是页面入口，不是所有 UI 的最终所在地

在 Next.js App Router 中，`app/page.tsx` 对应首页 `/`，`app/homework/page.tsx` 对应 `/homework`，`app/resources/page.tsx` 对应 `/resources`。很多 `page.tsx` 文件会很薄，只负责把页面级数据或组件接起来。真正复杂的界面通常在 `components/pages/*.tsx`。

URL	页面入口	主要组件
<code>/</code>	<code>app/page.tsx</code>	首页模块，学习地图、任务、资源等由 <code>home</code> 相关组件组成。
<code>/homework</code>	<code>app/homework/page.tsx</code>	<code>components/pages/HomeworkPracticeBoard.tsx</code>
<code>/resources</code>	<code>app/resources/page.tsx</code>	<code>components/pages/ResourcesBoard.tsx</code>
<code>/projects</code>	<code>app/projects/page.tsx</code>	<code>components/pages/ProjectsBoard.tsx</code>
<code>/admin</code>	<code>app/admin/page.tsx</code>	<code>components/pages/AdminBoard.tsx</code>
<code>/login</code>	<code>app/login/page.tsx</code>	登录页面和本地登录入口。

3.3 `components/pages` 是大页面的操作台

`components/pages` 里放的是页面主体。比如 `HomeworkPracticeBoard` 负责 Homework 页的选择器、题目详情、状态 chip、进度面板和老师站状态优先级。`ResourcesBoard` 负责 Lecture、Lab、Class Notes 等资料视图。`AdminBoard` 负责管理员日期表、文件上传、预览、保存和重置。

这种拆法的好处是 `page.tsx` 不会变成巨型文件。页面入口只负责路由层面，真正的用户交互和 UI 状态放到对应的 Board 组件里。以后要改 Homework 界面，先看 `HomeworkPracticeBoard`；要改 Admin，先看 `AdminBoard`。

3.4 Navbar 不是普通菜单，它连接了全站操作

`components/layout/Navbar.tsx` 负责顶部导航。它不只是几个链接，还管理 Week Navigation、全局搜索、移动端 drawer、主题切换位置、折叠状态和登出入口。根据 `.agentdocs/frontend/ui-design.md`，第一行应该是 Week Navigation、周切换和全局搜索，不能退回旧的只有品牌名的 header。

全局搜索的规则是：如果 URL 里有 `q` 参数，就优先用 URL 的 `q`；否则用本地保存的搜索词；都没有才是空搜索。用户在非首页提交搜索时，会跳回 `/?q=...`，让首页统一过滤模块。

3.5 CourseUiProvider 保存跨页面 UI 状态

`providers/CourseUiProvider.tsx` 是一个 React context。它保存 `selectedWeek`、`themeMode`、`globalQuery`、`completedTaskIds`、`bookmarkedResourceIds`、`questionProgressMap`、`weekNavCollapsed` 等状态。换句话说，它是全站 UI 偏好的小型本地状态中心。

这里有一个很关键的实现细节：它使用 `useSyncExternalStore` 读取 `localStorage`，而不是在 `useState` 初始化时直接读 `window.localStorage`。原因是 Next.js 有服务端渲染和客户端 hydration，直接在初始化时读浏览器对象容易造成服务端和客户端初始 HTML 不一致。

说明：本地状态不是老师站成绩。`questionProgressMap` 只能作为本地笔记或 fallback。Homework 的真实 JPA 状态应该优先使用 `/api/homework/status` 从老师站同步回来的结果。

3.6 样式系统：Tailwind 加 CSS token，而不是每个页面随便写颜色

这个项目使用 Tailwind CSS v4 和 `app/globals.css` 中的主题 token。页面里常见的 `surface-card`、`module tone`、`light/dark token` 都是为了让不同页面看起来属于同一个系统。Admin 页面即使优化成密集教授工作台，也应该继续复用现有 `surface-card`、薄边框、模块色调和响应式规则。

移动端和桌面端也不是简单缩放。Navbar 在较窄宽度下要用 drawer，避免导航挤在一行；移动端标签要 `min-w-0` 并允许截断，图标不要被压扁；表格和卡片要保证文字不重叠。

3.7 Home 页面怎么组成

Home 是课程总览页。它把 Week Navigation 选中的周、学习地图、任务日历、资源入口、Quiz 状态和搜索过滤组合起来。Home 的一些数据来自本地课程结构，一些来自和 Homework 共享的 Quiz API。

Home 的一个重要设计约束是 Task Calendar 嵌在 This Week Tasks 面板里，而不是独立的大块。AI Reflection 和 BK Reflection 是 `recommended due items`，Quiz 是 `required Sunday 23:59`，展示上要能看出 `required` 和 `recommended` 的区别。

3.8 Homework 页面怎么组成

Homework 页面不是只展示静态作业列表。它统一承载本地 JPA Homework、AI Reflection、BK Reflection 和 Quiz。`lib/practice/content.ts` 会把这些不同来源的任务适配成统一的选择器模型，`HomeworkPracticeBoard` 再把它们渲染出来。

JPA Homework 的题目详情来自 `data/homework/manifest.json` 和 `lib/homework/content.ts`。真实完成状态来自 `/api/homework/status`。显示时必须记住：老师站状态优先，本地浏览器 progress 只是 fallback 或个人笔记。

3.9 Resources 页面怎么组成

Resources 是 Lecture、Lab、Class Notes 等材料的工作台。Lecture 和 Lab 读取 `data/course-materials/manifest.json`，并通过 `/resources/materials/[id]` 预览。Class Notes 是管理员上传的本地材料，学生端通过 `/api/class-notes/week/[week]` 读取可见条目，通过 `/resources/class-notes/[id]` 预览。

课程 PDF 不应该直接指向 OneDrive 旁边的原始目录。真正给网页使用的资源应该在 `public/course-materials/` 下，`manifest` 里记录结构化信息。这样部署到 VCM 后，服务器也能找到这些文件。

3.10 Admin 页面怎么组成

Admin 是本地门户编辑器，不是老师站后台。它通过 `CS201_ADMIN_USERS` 控制谁能进，保存的是本地 JSON override，不会写老师站 BK、Reflection、Vote 或 JPA 数据。

Admin 的长期设计方向是教授工作台：按周管理日期、标题、隐藏状态、是否显示 due；管理 Class Notes 上传、可见性、预览和删除。`description/detail` 不应该再由 Admin 编辑，因为学生页面的基础说明仍来自课程源数据，Admin 只负责实际会变的本地展示控制。

4. 后端怎么工作

4.1 route.ts 是后端入口

在这个项目里，`app/api/.../route.ts` 就是 Next.js 的后端 API。文件名 `route.ts` 不是普通组件，它导出 GET、POST、PATCH、DELETE 等函数。当浏览器请求对应 URL 时，Next.js 在服务器上执行这些函数。

一个典型 API route 的结构是：先读 `request` 和 `params`，再验证 `session`，再检查参数合法性，然后调用 `lib` 模块，最后用 `NextResponse.json` 返回 JSON。这样 `route.ts` 保持薄，复杂逻辑留给 `lib`。

API 路径	route 文件	作用
<code>/api/auth/login</code>	<code>app/api/auth/login/route.ts</code>	本地账号登录，验证后写 <code>portal session cookie</code> 。
<code>/api/homework/status</code>	<code>app/api/homework/status/route.ts</code>	读取当前用户老师站 JPA 提交状态，并返回 sanitized status。
<code>/api/quiz/week/[week]</code>	<code>app/api/quiz/week/[week]/route.ts</code>	读取指定周 Quiz 列表和题目数据，不泄露正确答案。
<code>/api/reflections/week/[week]</code>	<code>app/api/reflections/week/[week]/route.ts</code>	读取指定周 Reflection 问卷状态和内容。
<code>/api/bk-projects/week/[week]</code>	<code>app/api/bk-projects/week/[week]/route.ts</code>	读取 BK Projects 小组、survey 和投票相关数据。
<code>/api/admin/overrides</code>	<code>app/api/admin/overrides/route.ts</code>	管理员保存本地课程覆盖数据。
<code>/api/admin/class-notes/upload</code>	<code>app/api/admin/class-notes/upload/route.ts</code>	管理员上传 Class Notes 文件。

4.2 lib/auth/session.ts 管本地登录 cookie

`lib/auth/session.ts` 定义了本地 `portal session` 的规则。登录成功后，服务器创建一个带签名的 token，放进名为 `cs201_portal_session` 的 cookie。后续请求带着这个 cookie，服务器就能验证用户是不是已经登录。

这个 token 不是随便拼的字符串。它包含 `username`、`issuedAt`、`authSource`，并用 `CS201_PORTAL_SESSION_SECRET` 做 HMAC-SHA256 签名。服务器验证签名、过期时间和用户是否仍在配置中，验证通过才认为 `session` 有效。

说明：文档不能写真实 `CS201_PORTAL_USERS`、`CS201_PORTAL_PASSWORD`、`CS201_PORTAL_SESSION_SECRET`。这里只说明机制，不记录任何真实值。

4.3 proxy.ts 是站点门卫

proxy.ts 在请求进入页面或 API 之前做统一保护。它允许 /login、/favicon.ico、/_next 等公开路径通过；其他页面如果没有有效 session，就重定向到 /login，或者在 teacher SS0 强制开启时跳到 /api/auth/teacher/start。

它还处理 restricted local users。受限测试账号可以浏览页面，但不能下载 /course-materials 和 /reflection-templates，也不能调用写入类 API 或启动 teacher SS0。这是为了让本地演示账号能看网站，但不能拿到受限课程文件或执行写操作。

4.4 teacher SS0 和老师站 token 为什么必须在服务器端

老师站 token 是访问外部老师站 API 的凭证。它不能返回给浏览器，也不能放进前端 localStorage。这个项目通过 lib/auth/teacherSso.ts 和相关 API 处理 teacher session。/api/auth/teacher/status 可以返回是否配置、用户元数据等非敏感信息，但不能返回 access token。

当用户通过老师站 SS0 登录时，Homework、Quiz、Reflection、BK Projects 可以优先使用 teacher session。没有 teacher session 时，某些模块才根据 CS201_BK_USER_MAP 或 CS201_QUIZ_USER_MAP 走本地 fallback。这个设计保证逻辑能泛化到当前登录用户，而不是硬编码某一个学生。

4.5 Homework JPA 状态同步

lib/homework/status.ts 是 Homework 老师站 JPA 状态同步的核心。它会先解析当前老师站 profile，得到 username；再请求 /api/user_courses_bk_jpa/{username}/ 找到 CS201 course id；然后请求 /api/user_jpa_submissions/{username}/{course_id}/ 获取 submissions；最后把 submissions 映射回本地 homework manifest 的每道题。

状态判断规则是最新 submit 决定结果，而不是最新有分数的 submit。也就是说，同一题如果先提交过 grades: 1，后来又提交 grades: 0，最新 submit 是 0，就应该显示 incorrect。最新 submit grade 缺失或 null，就是 ungraded。没有 submit row 才是 not-started。

老师站最新 submit 情况	门户显示
grade 或 grades 大于 0	correct
grade 或 grades 等于 0	incorrect
最新 submit 没有 grade 或 grade 是空	ungraded
没有任何 submit row	not-started

4.6 Quiz 后端为什么要保存 submit context

Quiz 页面不能只把题目 HTML 丢给前端。老师站的题目页面可能有 CSRF token、form action、答案字段顺序和随机化内容。后端 fetchParsedQuizProblem 解析题目时，会创建 server-only submit context，并只把一个不敏感的 submitContextId 给浏览器。

提交时，前端带着 submitContextId 和用户答案请求 /api/quiz/question/[problemId]/submit。后端用之前保存的 context 还原老师站表单上下文，保证提交使用的是学生刚刚看到的那一版题目顺序和 token。这样既能提交，又不会把老师站 cookie、CSRF 细节或正确答案暴露给浏览器。

4.7 Reflection 和 BK Projects 的后端同步

`lib/reflections/*` 管 Reflection: 包括老师站登录、课程树发现、问卷读取、`fallback` 数据和提交。

`lib/bk-projects/*` 管 BK Projects: 包括小组、`survey`、`vote` 等数据同步。它们复用同样的安全思路: 登录和 `token` 在服务器端, 返回给前端的是清洗后的展示数据。

手动 `Submit` 是这些模块唯一会写老师站的动作。普通读取页面时, 更多是 `fetch` 当前状态、显示已提交内容或可用表单。这样能减少误写外部系统的风险。

4.8 Admin 后端只写本地数据

`lib/admin/overrides.ts` 管课程覆盖数据。它读取和写入 `data/admin-overrides/course-overrides.json`。`sanitizeCourseOverrides` 会检查 `payload` 必须是对象, `title` 必须是字符串, `dueDate` 必须是合法日期, `hidden/showDue` 必须是布尔值。

`Class Notes` 由 `lib/class-notes/store.ts` 管。它允许 `PDF`、`PNG`、`JPG`、`JPEG`、`WEBP`, 检查扩展名和 `MIME` 是否匹配, 文件写入 `public/course-materials/class-notes/`, 元数据写入 `data/admin-overrides/class-notes.json`。学生端只读取没有 `hidden` 的 `note`。

说明: Admin 保存的本地数据会影响学生门户展示, 但它不等于老师站发布。保存日期和隐藏状态不会改老师站原始作业、Quiz、Reflection 或 BK Project。

5. 关键业务流程逐步解释

5.1 本地登录流程

1. 用户访问受保护页面, 例如 `/homework`。
2. `proxy.ts` 发现没有 `cs201_portal_session` cookie, 于是跳到 `/login`, 并带上 `next` 参数。
3. 用户在登录页输入本地账号密码。
4. 前端提交到 `/api/auth/login`。
5. `app/api/auth/login/route.ts` 调用 `lib/auth/session.ts` 检查 `CS201_PORTAL_USERS` 或 `legacy username/password` 配置。
6. 验证通过后, 后端创建签名 `session token`, 写入 `cs201_portal_session` cookie。
7. 浏览器带着 `cookie` 回到原来的 `next` 页面。

5.2 teacher SS0 登录流程

1. 当 `CS201_REQUIRE_TEACHER_SS0=1` 且 `SS0` 配置完整时, 未登录用户会被 `proxy.ts` 送到 `/api/auth/teacher/start`。
2. `start route` 生成 `state`, 把用户带到老师站授权页。
3. 老师站回调 `/api/auth/teacher/callback`, 服务器交换 `token`。
4. 服务器创建 `authSource` 为 `teacher` 的本地 `portal session`, 同时保存服务器端可用的 `teacher session` 信息。
5. 后续 `Homework`、`Quiz`、`Reflection`、`BK Projects` 优先用这个 `teacher session` 同步当前用户数据。

5.3 打开 Homework 并看到真实 JPA 状态

1. 浏览器打开 `/homework`, `Next.js` 加载 `app/homework/page.tsx`。

2. 页面渲染 HomeworkPracticeBoard, 先展示本地 homework manifest 里的作业和题目。
3. 组件请求 /api/homework/status。
4. API 验证 portal session。如果有 teacher session, 就用 teacher token; 否则尝试 CS201_BK_USER_MAP fallback。
5. lib/homework/status.ts 获取当前老师站 profile、CS201 course id 和 JPA submissions。
6. 后端按最新 submit 规则计算每道题 correct、incorrect、ungraded、not-started。
7. 前端把老师站状态显示在 homework selector、question selector、detail badge 和 progress panel 上。
8. 如果老师站同步不可用, 前端才回退到本地 questionProgressMap, 不把本地笔记当成真实成绩。

5.4 提交 Quiz 的流程

1. 用户进入 Homework 页里的 Quiz 项, 或 Home 页查看 Quiz 状态。
2. 前端请求 /api/quiz/week/[week]。
3. 后端用 teacher session 或 quiz credential 登录老师站 Quiz 系统, 解析 assignment 页面和题目页面。
4. 后端把 prompt、answerFields、status、submitContextId 返回前端, 但不返回正确答案。
5. 用户填写答案并提交。
6. 前端请求 /api/quiz/question/[problemId]/submit, 带上 submitContextId 和答案。
7. 后端使用 server-only submit context 把表单提交给老师站。
8. 提交完成后, 前端刷新本周 Quiz 数据, 确保状态和老师站一致。

5.5 同步 Reflection 的流程

1. 用户在 Homework 中选择 AI Reflection 或 BK Reflection。
2. 前端请求 /api/reflections/week/[week]。
3. 后端验证 session, 使用 teacher session 或 CS201_BK_USER_MAP fallback。
4. lib/reflections 读取老师站对应周的问卷、提交状态和可展示内容。
5. 前端显示推荐 due date、问卷内容、已有提交或手动提交入口。
6. 用户手动 Submit 时, 后端才把文本写回老师站。

5.6 Resources 预览材料的流程

1. ResourcesBoard 读取材料列表, Lecture/Lab 来自 data/course-materials/manifest.json, Class Notes 来自 /api/class-notes/week/[week]。
2. 用户点击 Lecture 或 Lab 时, 进入 /resources/materials/[id]。
3. 用户点击 Class Notes 时, 进入 /resources/class-notes/[id]。
4. 页面根据 id 找到对应文件和元数据, 再展示 PDF 或图片预览。
5. 受限用户或未登录用户不能直接绕过页面下载 protected course material, 因为 proxy.ts 会拦截相关路径。

5.7 Admin 上传 Class Notes 的流程

1. 管理员进入 /admin, 页面由 AdminBoard 渲染。

2. 管理员选择一个或多个 PDF/PNG/JPG/JPEG/WEBP 文件。
3. 前端为每个文件生成上传队列项，默认 week 是当前选中周，标题可以从清理后的文件名自动生成。
4. 前端把文件发给 `/api/admin/class-notes/upload`。
5. 后端检查管理员权限、检查扩展名和 MIME 是否匹配。
6. 文件保存到 `public/course-materials/class-notes/`，元数据保存到 `data/admin-overrides/class-notes.json`。
7. 上传成功后，Preview as student 只显示没有 hidden 的 Class Notes。

5.8 Admin 保存日期和可见性覆盖的流程

1. 管理员在 Dates 表格里修改 title、date、time、hidden、show due。
2. 前端标记 unsaved changes，但不会自动发布。
3. 管理员点击 Save。
4. 前端请求 `/api/admin/overrides`。
5. 后端检查 CS201_ADMIN_USERS 权限，调用 `sanitizeCourseOverrides` 清洗 payload。
6. 合法数据写入 `data/admin-overrides/course-overrides.json`，并记录 `updatedAt`、`updatedBy`。
7. 学生页面之后读取课程数据时，会应用这些本地 overrides。

6. 数据、文件和安全

6.1 本地 JSON 分几类

这个项目没有引入数据库，主要靠本地 JSON 和 public 文件组织课程数据。这样部署简单，也方便课程门户这种读多写少的场景。但因为没有数据库事务和后台管理系统，所有写入路径都必须谨慎清洗和测试。

文件	作用	是否可手动乱改
<code>data/course-site/normalized.json</code>	导入后的官方 CS201 course-site 结构化数据	不建议。应通过 <code>npm run import:course-site</code> 刷新。
<code>data/homework/manifest.json</code>	JPA Homework 和题目详情 manifest	不建议。应通过 <code>npm run build:homework</code> 生成。
<code>data/course-materials/manifest.json</code>	Lecture/Lab 等课程材料 manifest	不建议。应通过 <code>npm run build:materials</code> 生成。
<code>data/admin-overrides/course-overrides.json</code>	Admin 保存的标题、日期、隐藏和 due 展示覆盖	可以由 Admin API 写，不建议手写。
<code>data/admin-overrides/class-notes.json</code>	Class Notes 元数据	可以由 Admin API 写，不建议手写。

6.2 public 目录为什么能被浏览器访问

`public/` 下的文件会被 `Next.js` 当作静态资源服务。比如 `public/course-materials/...` 最终可以通过 `/course-materials/...` 访问。这个特性很方便，但也意味着只要放进 `public`，就要配合登录保护和访问规则，不能把不该公开的文件随便暴露。

Class Notes 文件保存在 `public/course-materials/class-notes/`。这并不等于任何人都能直接拿，因为 `proxy.ts` 会拦截 `/course-materials` 路径并检查登录和受限用户规则。

6.3 .env.local 是秘密边界

.env.local 里放运行时配置和秘密，例如本地登录用户、session secret、老师站 base URL、用户映射、SSO 配置等。文档、前端代码、GitHub 仓库都不能记录真实值。前端组件也不能读取这些秘密；只有服务器端 route 和 lib 可以通过 process.env 使用它们。

环境变量类型	说明
CS201_PORTAL_USERS 或 legacy portal username/password	本地门户账号配置，只能在服务器端验证。
CS201_PORTAL_SESSION_SECRET	签名 portal session cookie 的 secret，生产环境必须配置。
CS201_BK_USER_MAP	本地 fallback: portal 用户到老师站用户的映射，服务器端使用。
CS201_QUIZ_USER_MAP	Quiz fallback 映射，服务器端使用。
CS201_REQUIRE_TEACHER_SSO	控制是否强制未登录用户走 teacher SSO。
CS201_ALLOW_INSECURE_COOKIES	VCM HTTP 环境需要的 cookie 兼容开关，HTTPS 部署通常不应开启。

6.4 为什么前端不能拿老师站 token

浏览器里的 JavaScript 可以被用户查看、调试和复制。如果把老师站 token 发给前端，就等于把可以代表用户访问老师站的钥匙交给了浏览器。正确做法是：前端只拿 submitContextId、status、prompt、title 这些不敏感展示数据；老师站 token、cookie、CSRF 细节、真实 credentials 留在服务器端。

这个原则也解释了为什么 Quiz 不返回正确答案，Reflection/BK Projects 不返回原始 credential，Homework status 只返回 sanitized status metadata。门户的目标是帮学生看到自己的状态和提交入口，不是复制老师站后台。

6.5 本地浏览器状态和服务器状态要分清

CourseUiProvider 保存的 selectedWeek、themeMode、bookmark、local progress 都是用户浏览器本地状态。这些东西让界面更好用，但不是权威课程数据。老师站返回的 Homework/Quiz/Reflection 状态才是外部系统里的真实状态。本地状态可以辅助显示，不能覆盖老师站结果。

7. 部署和验证

7.1 本地开发怎么跑

本地开发通常用 npm run dev。它启动 Next.js dev server，适合边改边看。这个仓库路径在 OneDrive 和中文目录下，历史上 Next.js/Turbopack 和后台进程可能出现路径或权限问题，所以如果遇到 dev server 不稳定，要同时检查端口、HTTP 响应和进程是否真的还活着。

命令	作用
npm run dev	启动开发服务器，适合本地调试。
npm run build	生产构建，检查 Next.js 编译、类型和打包问题。
npm run start -- --hostname 0.0.0.0 --port 3300	启动生产服务器，VCM systemd 使用这个形式。
npm run build:materials	根据课程材料源刷新 data/course-materials/manifest.json 和 public 资源。
npm run build:homework	根据导入的 course-site raw HTML 刷新 Homework manifest。
npm run import:course-site	用环境变量中的凭证导入官方 course-site 内容。

7.2 Duke VCM 生产部署怎么理解

当前 Duke VCM 部署运行在 `vcm-53362.vm.duke.edu`，服务名是 `cs201-portal.service`，端口是 3300。生产启动命令是 `npm run start -- --hostname 0.0.0.0 --port 3300`。`systemd` 负责让服务开机启动、崩溃后重启，以及统一查看状态。

版本化部署的思路是不要覆盖老的 `/opt/cs201-portal/app`。新版本放到 `/opt/cs201-portal/releases/<timestamp>-vN`，再让 `/opt/cs201-portal/current` 指向当前版本。回滚时只要把 `current` 指回旧版本并重启服务。

7.3 测试命令分别检查什么

命令	检查对象	适合发现的问题
<code>npm run lint</code>	ESLint	语法风格、React/Next 规则、潜在错误。
<code>npm run test</code>	Vitest 单元和组件测试	parser、状态计算、组件渲染、API 边界逻辑。
<code>npm run test:e2e</code>	Playwright 真实浏览器测试	登录跳转、页面可见性、按钮交互、移动端/桌面端渲染。
<code>curl</code> 或 <code>Invoke-WebRequest</code> <code>t</code>	真实 HTTP 响应	服务是否真的能访问，而不是只有端口在监听。
<code>systemctl status</code> <code>cs201-portal.service</code>	VCM 进程和日志	生产服务是否启动、是否崩溃、工作目录是否正确。

7.4 排查问题时不要只看一个现象

比如用户说网站打不开，不能只看 `npm` 打印了 `Ready`。还要用浏览器或 `Invoke-WebRequest` 检查真实 URL 是否返回 HTML；用 `netstat` 或 `Get-Process` 看端口是否还活着；如果是 VCM，则用 `systemctl` 和 `curl 127.0.0.1:3300` 检查服务器内部是否通。

比如用户说 `Homework` 状态不对，不能只看本地 `progress`。要检查 `/api/homework/status` 返回的 `questionStatuses`、`homeworkStatuses`、`syncStatus`、`username`、`courseId`；再确认 `lib/homework/status.ts` 是否按最新 `submit` 规则计算。

8. 以后维护时怎样定位问题

8.1 先按页面定位，再按数据流定位

最简单的定位方式是两步。第一步，问题出现在哪个页面，就先找 `app/.../page.tsx` 和 `components/pages/...Board.tsx`。第二步，这个页面的数据从哪里来，就找对应 `app/api/.../route.ts` 和 `lib/...` 模块。

用户看到的问题	先看前端	再看后端或数据
Home 周任务或搜索不对	<code>components/home/*</code> ， <code>app/page.tsx</code>	<code>lib/home/homeFilters.ts</code> ， <code>lib/course/*</code>
Homework 状态不对	<code>components/pages/HomeworkPracticeBoard.tsx</code>	<code>app/api/homework/status/route.ts</code> ， <code>lib/homework/status.ts</code>
Quiz 提交后状态没刷新	<code>HomeworkPracticeBoard Quiz</code> 相关逻辑	<code>app/api/quiz/*</code> ， <code>lib/quiz/*</code> ， <code>lib/quiz/submitContext.ts</code>
Reflection 内容不同步	<code>HomeworkPracticeBoard reflection</code> 视图	<code>app/api/reflections/*</code> ， <code>lib/reflections/*</code>

用户看到的问题	先看前端	再看后端或数据
Resources 材料缺失	components/pages/ResourcesBoard.tsx	data/course-materials/manifest.json, lib/course-materials/content.ts
Class Notes 上传或预览失败	components/pages/AdminBoard.tsx, ResourcesBoard.tsx	app/api/admin/class-notes/*, lib/class-notes/store.ts
Admin 日期没保存	components/pages/AdminBoard.tsx	app/api/admin/overrides/*, lib/admin/overrides.ts

8.2 判断一个功能属于前端还是后端

如果问题是按钮位置、文字显示、卡片密度、移动端重叠、状态 chip 颜色，那大概率是前端。去 components、providers、app/globals.css 看。

如果问题是登录权限、老师站同步、保存 JSON、上传文件、隐藏数据仍被返回、token 泄露风险，那大概率是后端。去 app/api、lib/auth、lib/homework、lib/quiz、lib/reflections、lib/admin、lib/class-notes 看。

如果问题是数据本身不完整，例如某周材料缺失、作业 manifest 里没有题目，那可能是脚本或本地数据生成问题。去 scripts、data/course-site、data/homework、data/course-materials 看。

附录 A. 小白词汇表

词	通俗解释
React	把网页拆成组件的 JavaScript UI 库。
Component	组件。可以理解成网页零件，例如 NavBar、TaskCard、HomeworkPracticeBoard。
Route	路由。URL 和代码文件之间的对应关系。/homework 对应 app/homework/page.tsx。
API	前端和后端约好的请求入口。浏览器请求 URL，服务器返回 JSON。
Cookie	浏览器随请求带给服务器的小数据。这里用来保存登录 session token。
Server-side	服务器端执行。可以读 env、访问老师站 token、写本地文件。
Client-side	浏览器端执行。负责交互展示，但不能保存秘密。
TypeScript	带类型的 JavaScript，帮助提前发现数据结构错误。
Tailwind CSS	用工具类快速写样式的 CSS 系统，本项目还配合 globals.css token。
Vitest	跑单元测试和组件测试的工具。
Playwright	自动打开真实浏览器测试网站的工具。
Manifest	清单文件。把课程材料、作业题目等结构化成 JSON，方便页面读取。
Override	覆盖数据。Admin 不改原始课程数据，而是写本地覆盖。
Teacher SSO	通过老师站登录并让门户服务器代表当前用户同步老师站数据。
JPA submission	老师站里 JPA 作业每道题的提交记录。

附录 B. 重要路径索引

路径	说明
app/layout.tsx	全站根布局, 挂载 provider、Navbar 和页面容器。
app/globals.css	全局样式、主题 token、surface-card 等视觉基础。
components/layout/Navbar.tsx	周导航、全局搜索、主导航、移动端 drawer。
providers/CourseUiProvider.tsx	当前周、主题、搜索、本地进度、收藏等 UI 状态。
components/pages/HomeworkPracticeBoard.tsx	Homework 主交互界面, 展示 JPA/Reflection/Quiz。
components/pages/ResourcesBoard.tsx	Resources 主界面, 展示 Lecture/Lab/Class Notes。
components/pages/AdminBoard.tsx	Admin 主界面, 管理日期覆盖和 Class Notes。
app/api/homework/status/route.ts	Homework JPA 状态 API 入口。
lib/homework/status.ts	老师站 JPA submission 解析和 latest submit 状态计算。
app/api/quiz/week/[week]/route.ts	Quiz 周数据 API。
lib/quiz/*	Quiz 登录、HTML 解析、submit context、提交结果解析。
app/api/reflections/week/[week]/route.ts	Reflection 周数据 API。
lib/reflections/*	Reflection 老师站同步、fallback、提交。
app/api/bk-projects/week/[week]/route.ts	BK Projects 周数据 API。
lib/bk-projects/*	BK Projects 小组、survey、vote 同步。
lib/auth/session.ts	本地 portal session cookie 创建和验证。
proxy.ts	登录保护、受限用户保护、SSO 跳转。
lib/admin/overrides.ts	Admin 本地课程覆盖 JSON 的清洗、读取、写入。
lib/class-notes/store.ts	Class Notes 文件校验、存储、元数据管理。
data/course-site/normalized.json	导入后的官方 course-site 结构化数据。
data/homework/manifest.json	生成的 Homework/JPA 题目清单。
data/course-materials/manifest.json	生成的 Lecture/Lab 课程材料清单。
data/admin-overrides/course-overrides.json	Admin 日期、标题、隐藏、showDue 覆盖数据。
data/admin-overrides/class-notes.json	Class Notes 元数据。
public/course-materials/	部署时给浏览器访问的课程材料文件。
.agentdocs/frontend/architecture.md	前端架构和跨模块约束, 改前端时必读。
.agentdocs/frontend/ui-design.md	界面设计约束, 改 UI 时必读。
.agentdocs/deployment/vcm.md	Duke VCM 部署路径、systemd、端口和验证命令。

附录 C. 一分钟讲解模板

如果要向别人快速解释这个网站，可以这样讲：CS201 Portal 是一个 Next.js 全栈课程门户。前端用 React 组件做页面，后端用 Next.js API routes 处理登录、权限、老师站同步和本地文件。学生看到的 Home、Homework、Resources、Projects 都是 React 组件；真实的 Homework/Quiz/Reflection 状态由服务器端 API 根据当前用户去老师站同步；Admin 只管理本地展示数据和 Class Notes，不改老师站。课程资料和导入数据主要存成本地 JSON 和 public 文件，生产环境由 Duke VCM 上的 systemd 服务在 3300 端口运行。

附录 D. 维护时的基本原则

1. 不要把 .env.local 的任何真实值写进文档、前端代码或提交记录。
2. 不要让前端直接拿老师站 token、cookie、CSRF 或正确答案。
3. 不要把本地 localStorage progress 当成老师站真实成绩。
4. 不要手写 public 文件路径去绕过 manifest，材料应通过生成脚本和 lib 内容入口读取。
5. 改页面先看 .agentdocs/frontend/architecture.md 和 ui-design.md，再看对应 Board 组件。
6. 改 API 先看 app/api 对应 route，再看 lib 业务模块和测试。
7. 改 Admin 时记住：它保存本地 overrides，不发布到老师站。
8. 任何涉及同步、登录、上传、权限的改动，都要跑 lint、test、test:e2e，并做浏览器渲染检查。